

Object-Oriented Programming (ENCT 151)

Complete Solved Question Bank

Tribhuvan University
Institute of Engineering (IOE)

Solutions compiled for Bachelor in Engineering (BE) Level

Contents

1	Introduction to Object-Oriented Programming	5
2	C++ Basics and Functions	9
3	Objects, Classes, and Friends	13
4	Operator Overloading	19
5	Inheritance	25
6	Virtual Functions and Polymorphism	31
7	Stream Computation and File Handling	37
8	Templates and STL	43
9	Exception Handling	47

Chapter 1

Introduction to Object-Oriented Programming

Question [2083 Baishakh] Q1

Problem: What are the advantages of object oriented programming? Differentiate between procedural oriented programming and object oriented programming.

Solution: Object-Oriented Programming (OOP) offers several advantages:

- **Modularity:** The source code for an object can be written and maintained independently of the source code for other objects.
- **Reusability:** Inheritance allows reusing existing code, reducing development time.
- **Data Hiding (Security):** Data encapsulation prevents unauthorized access to internal data from outside the class.
- **Polymorphism:** It allows a single function to behave differently based on the object, making code flexible.
- **Easy Maintenance:** OOP makes it easier to track and resolve bugs. Real-world modeling makes the system more understandable.

Differences between Procedural Oriented Programming (POP) and Object-Oriented Programming (OOP):

Procedural Oriented Programming (POP)	Object-Oriented Programming (OOP)
Emphasis is on procedures (functions).	Emphasis is on data rather than procedure.
Large programs are divided into smaller programs known as functions.	Programs are divided into what are known as objects.
Most of the functions share global data.	Data is hidden and cannot be accessed by external functions.
Data moves openly around the system from function to function.	Objects may communicate with each other through functions.
Follows top-down approach in program design.	Follows bottom-up approach in program design.
Examples: C, VB, FORTRAN, Pascal.	Examples: C++, Java, Python.

Question [2082 Bhadra] Q1

Problem: What is the drawback of procedural programming and advantage of object-oriented programming along with suitable program code of each?

Solution: Drawbacks of Procedural Programming:

- **Data Insecurity:** Global data is accessible from anywhere in the program, making it vulnerable to accidental modification.
- **Poor Modeling:** Real-world problems are poorly modeled because POP focuses on functions rather than data/entities.
- **Difficult to Maintain:** As programs grow larger, managing interconnected functions and global variables becomes very complex.

Advantages of Object-Oriented Programming: As explained in the previous question, OOP provides data security (encapsulation), reusability (inheritance), and maps better to real-world problems.

Program Code for Procedural Programming (C-style):

```
1 #include <iostream>
2 using namespace std;
3
4 int globalData = 10; // Global data, insecure
5
6 void modifyData() {
7     globalData = 20; // Any function can change it
8 }
9
10 int main() {
11     modifyData();
12     cout << "Global Data: " << globalData << endl;
13     return 0;
14 }
```

Program Code for Object-Oriented Programming (C++ style):

```
1 #include <iostream>
2 using namespace std;
3
4 class DataHandler {
5 private:
6     int secureData; // Hidden data, secure
7
8 public:
9     DataHandler() { secureData = 10; }
10
11     void modifyData() {
12         secureData = 20; // Only member functions can change it
13     }
14
15     void display() {
16         cout << "Secure Data: " << secureData << endl;
17     }
18 };
19
20 int main() {
21     DataHandler obj;
22     obj.modifyData();
23     obj.display();
24     return 0;
25 }
```

Question [2082 Baishakh] Q1

Problem: Compare procedure oriented programming and object-oriented programming. Explain the basic concept of object oriented programming.

Solution: The comparison between POP and OOP is detailed in the solution for [2083 Baishakh] Q1.

Basic Concepts of Object-Oriented Programming:

- **Object:** The basic runtime entity in an object-oriented system. It may represent a person, a place, a bank account, etc.
- **Class:** A blueprint or prototype that defines the variables and methods common to all objects of a certain kind. It is a user-defined data type.
- **Encapsulation:** The wrapping up of data and functions into a single unit (called class). It keeps data safe from outside interference.
- **Data Abstraction:** Providing only essential information to the outside world and hiding the background details or implementation.
- **Inheritance:** The process by which objects of one class acquire the properties of objects of another class. It supports the concept of hierarchical classification and reusability.
- **Polymorphism:** The ability to take more than one form. For example, an operation may exhibit different behaviors in different instances depending upon the types of data used.
- **Dynamic Binding:** The code associated with a given procedure call is not known until the time of the call at runtime.
- **Message Passing:** Objects communicate with one another by sending and receiving information, much like people pass messages to one another.

Question [2081 Ashwin] Q1

Problem: What are the benefits of object-oriented programming? Describe the feature of object-oriented programming.

Solution: The benefits of Object-Oriented Programming are detailed in the solution for [2083 Baishakh] Q1.

The features of Object-Oriented Programming, such as Objects, Classes, Encapsulation, Abstraction, Inheritance, Polymorphism, Dynamic Binding, and Message Passing, are detailed in the solution for [2082 Baishakh] Q1.

Chapter 2

C++ Basics and Functions

Question [2083 Baishakh] Q2

Problem: When do we use default argument in functions? When can conflict occur when default argument and function overloading are used together? Create a C++ program to illustrate the concept of default arguments. Define a function named `printMessage` that takes two parameters: a message and a count. The function should print the string for count number of times. Set the default value of count to 1.

Solution: When to use default arguments: Default arguments are used when a function is frequently called with the same value for a particular parameter. It simplifies function calls by allowing the caller to omit that argument, in which case the compiler automatically supplies the default value.

Conflict between Default Argument and Function Overloading: A conflict (ambiguity) occurs when an overloaded function can be called in a way that matches multiple signatures due to default arguments. For example, if we have:

```
1 void display(int a);  
2 void display(int a, int b = 10);
```

Calling `display(5)` is ambiguous because the compiler doesn't know whether to call the first function or the second function (using the default argument).

C++ Program:

```
1 #include <iostream>  
2 #include <string>  
3 using namespace std;  
4  
5 // Function with default argument  
6 void printMessage(string message, int count = 1) {  
7     for (int i = 0; i < count; i++) {  
8         cout << message << endl;  
9     }  
10 }  
11  
12 int main() {  
13     // Calling with both arguments  
14     cout << "Calling printMessage(\"Hello\", 3):" << endl;  
15     printMessage("Hello", 3);  
16  
17     // Calling with only the required argument  
18     cout << "\nCalling printMessage(\"Welcome\"): " << endl;  
19     printMessage("Welcome");  
20  
21     return 0;  
22 }
```

Question [2082 Bhadra] Q2

Problem: What do you mean by Namespace? Why namespace feature is needed in C++? Explain with example how object can be passed or returned from functions.

Solution: Namespace: A namespace is a declarative region that provides a scope to the identifiers (the names of types, functions, variables, etc.) inside it. It helps organize code into logical groups.

Why namespace is needed: In large projects, multiple libraries might use the same name for a class or a function (e.g., a `display()` function). This causes a naming collision. Namespaces solve this problem by isolating identifiers so that `LibraryA::display()` and `LibraryB::display()` can coexist without conflict.

Objects Passed or Returned from Functions: Objects can be passed to functions by value, by reference, or by pointers, similar to built-in data types. A function can also return an object.

```
1 #include <iostream>
2 using namespace std;
3
4 class Number {
5 public:
6     int val;
7     Number(int v = 0) { val = v; }
8 };
9
10 // Object passed by reference and object returned
11 Number addNumbers(Number& n1, Number& n2) {
12     Number temp;
13     temp.val = n1.val + n2.val;
14     return temp; // Returning an object
15 }
16
17 int main() {
18     Number num1(10), num2(20);
19     Number result = addNumbers(num1, num2); // Passing objects
20
21     cout << "Sum: " << result.val << endl;
22     return 0;
23 }
```

Question [2082 Baishakh] Q2

Problem: When should you prefer the use of inline function? Explain with example. Explain function overloading in object-oriented programming.

Solution: Inline Function: An inline function is a function that is expanded in line when it is invoked. That is, the compiler replaces the function call with the corresponding function code. **When to prefer inline functions:** Inline functions are preferred for small, frequently called functions. They eliminate the overhead of a function call (like saving context, pushing arguments to the stack), thereby increasing execution speed. However, for large functions, inlining can cause code bloat, increasing the size of the executable.

```
1 #include <iostream>
2 using namespace std;
3
4 // Inline function
5 inline int square(int x) {
6     return x * x;
7 }
8
9 int main() {
```

```

10     cout << "Square of 5 is: " << square(5) << endl;
11     // The compiler replaces square(5) with (5 * 5)
12     return 0;
13 }

```

Function Overloading: Function overloading is a feature in C++ where multiple functions can have the same name but different parameters (different number of arguments, different types of arguments, or both). The compiler determines which function to call based on the arguments passed. It is an example of compile-time polymorphism.

Question [2081 Ashwin] Q2

Problem: Define reference variable and write syntax for defining reference variable. WAP a function which calculates A where $A = P(1 + r/100)^n$ for data p, r and n. But for some cases value of r is fix that is 50. Call this function in main.

Solution: Reference Variable: A reference variable is an alias, that is, another name for an already existing variable. Once a reference is initialized with a variable, either the variable name or the reference name may be used to refer to the variable.

Syntax: data_type &reference_name = variable_name;

C++ Program:

```

1  #include <iostream>
2  #include <cmath>
3  using namespace std;
4
5  // Function using default argument for r
6  double calculateA(double p, double n, double r = 50.0) {
7      return p * pow((1 + r / 100.0), n);
8  }
9
10 int main() {
11     double p = 1000.0;
12     double n = 2.0;
13
14     // Calling without passing r (uses default r = 50)
15     double result1 = calculateA(p, n);
16     cout << "Amount with default rate (50%): " << result1 << endl;
17
18     // Calling and explicitly passing r
19     double custom_r = 10.0;
20     double result2 = calculateA(p, n, custom_r);
21     cout << "Amount with custom rate (10%): " << result2 << endl;
22
23     return 0;
24 }

```


Chapter 3

Objects, Classes, and Friends

Question [2083 Baishakh] Q3

Problem: What do you mean by a constructor? Explain with example types of constructors in C++. Create a class person with data members name, age, address and citizenship_number. Define constructor to initialize the value of the person. Assign citizenship number if the age of the person is greater than 16 otherwise assign value zero to citizenship number. Also create a function to display the values.

Solution: Constructor: A constructor is a special member function of a class which has the same name as that of the class. It is automatically invoked whenever an object of its associated class is created. Its primary purpose is to initialize the object's data members. Constructors do not have a return type.

Types of Constructors:

- **Default Constructor:** A constructor with no parameters.
- **Parameterized Constructor:** A constructor that takes arguments to initialize objects with specific values.
- **Copy Constructor:** A constructor that initializes an object using another object of the same class.

C++ Program:

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class Person {
6 private:
7     string name;
8     int age;
9     string address;
10    long citizenship_number;
11
12 public:
13     // Parameterized constructor
14     Person(string n, int a, string addr, long c_num) {
15         name = n;
16         age = a;
17         address = addr;
18         if (age > 16) {
19             citizenship_number = c_num;
20         } else {
21             citizenship_number = 0;
22         }
23     }
24 }
```

```

23     }
24
25     void display() {
26         cout << "Name: " << name << endl;
27         cout << "Age: " << age << endl;
28         cout << "Address: " << address << endl;
29         cout << "Citizenship Number: " << citizenship_number << endl;
30         cout << "-----" << endl;
31     }
32 };
33
34 int main() {
35     Person p1("Ram", 25, "Kathmandu", 123456789);
36     Person p2("Sita", 14, "Lalitpur", 987654321);
37
38     p1.display();
39     p2.display();
40
41     return 0;
42 }

```

Question [2082 Bhadra] Q2 (Part 2)

Problem: In which condition can a class be made friend? How can we make a specific member function of a class a friend function? With a complete example show the both of these cases of friend class and friend function of another class.

Solution: When to make a class a friend: A class can be made a friend of another class when it needs to access the private or protected members of that other class. This is usually done when two classes are closely related and must cooperate closely (e.g., a LinkedList class and a Node class).

Making a specific member function a friend: We can declare a specific member function of one class as a friend of another class. We do this by using the scope resolution operator :: to specify the function belonging to the other class.

C++ Program:

```

1  #include <iostream>
2  using namespace std;
3
4  class ClassB; // Forward declaration
5
6  class ClassA {
7  public:
8      void showSecretB(ClassB& b); // Member function
9  };
10
11 class ClassB {
12 private:
13     int secretVal = 42;
14
15     // Making a specific member function of ClassA a friend
16     friend void ClassA::showSecretB(ClassB& b);
17
18     // Making an entire class a friend
19     friend class ClassC;
20 };
21
22 // Definition of ClassA's member function
23 void ClassA::showSecretB(ClassB& b) {
24     cout << "ClassA specific function accessing ClassB's secret: " << b.
25         secretVal << endl;

```

```

26
27 class ClassC {
28 public:
29     void accessB(ClassB& b) {
30         cout << "ClassC accessing ClassB's secret (Friend Class): " << b.
secretVal << endl;
31     }
32 };
33
34 int main() {
35     ClassB bObj;
36     ClassA aObj;
37     ClassC cObj;
38
39     aObj.showSecretB(bObj);
40     cObj.accessB(bObj);
41
42     return 0;
43 }

```

Question [2082 Baishakh] Q3

Problem: Write about different types constructors with respective syntaxes. In which condition can a class be made friend? WAP to create a class information to store the name and address of students. Store the information of two students and swap the values of these objects.

Solution: The types of constructors and conditions for friend class have been explained in [2083 Baishakh Q3] and [2082 Bhadra Q2].

Syntaxes for Constructors:

- Default: `ClassName() { ... }`
- Parameterized: `ClassName(type1 arg1, type2 arg2) { ... }`
- Copy: `ClassName(const ClassName &obj) { ... }`

C++ Program (Swapping objects):

```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class Information {
6 private:
7     string name;
8     string address;
9
10 public:
11     Information(string n = "", string addr = "") {
12         name = n;
13         address = addr;
14     }
15
16     void display() {
17         cout << "Name: " << name << ", Address: " << address << endl;
18     }
19
20     // Friend function to swap objects
21     friend void swapStudents(Information& s1, Information& s2);
22 };
23
24 void swapStudents(Information& s1, Information& s2) {

```

```

25     Information temp = s1;
26     s1 = s2;
27     s2 = temp;
28 }
29
30 int main() {
31     Information student1("Hari", "Pokhara");
32     Information student2("Shyam", "Chitwan");
33
34     cout << "Before Swapping:" << endl;
35     student1.display();
36     student2.display();
37
38     swapStudents(student1, student2);
39
40     cout << "\nAfter Swapping:" << endl;
41     student1.display();
42     student2.display();
43
44     return 0;
45 }

```

Question [2081 Ashwin] Q3

Problem: Explain friend function and with program code, how friend function can be used to make bridge between two classes? Create a class time, constructor having hour, minutes and second as arguments is used to initialize time objects. The add function that takes two class objects as arguments add them respectively then display the aggregate result. (Apply 60 seconds = 1 minute, 60 minutes = 1 hour and 24 hour = 1 day).

Solution: Friend Function: A friend function is a function that is not a member of a class but has access to its private and protected members. It is declared inside the class using the friend keyword.

Bridge between two classes: When a function needs to access private data from two different, unrelated classes, it can be declared as a friend in both classes. This function acts as a bridge between them.

C++ Program (Time Class):

```

1  #include <iostream>
2  using namespace std;
3
4  class Time {
5  private:
6      int days, hours, minutes, seconds;
7
8  public:
9      Time(int h = 0, int m = 0, int s = 0) {
10         days = 0;
11         hours = h;
12         minutes = m;
13         seconds = s;
14         normalize();
15     }
16
17     void normalize() {
18         minutes += seconds / 60;
19         seconds %= 60;
20
21         hours += minutes / 60;
22         minutes %= 60;
23     }

```

```

24     days += hours / 24;
25     hours %= 24;
26 }
27
28 void add(Time t1, Time t2) {
29     seconds = t1.seconds + t2.seconds;
30     minutes = t1.minutes + t2.minutes;
31     hours = t1.hours + t2.hours;
32     days = t1.days + t2.days;
33     normalize();
34 }
35
36 void display() {
37     cout << days << " Days, " << hours << " Hours, "
38         << minutes << " Minutes, " << seconds << " Seconds." << endl;
39 }
40 };
41
42 int main() {
43     Time t1(20, 45, 50);
44     Time t2(10, 30, 40);
45
46     Time result;
47     result.add(t1, t2);
48
49     cout << "Time 1: "; t1.display();
50     cout << "Time 2: "; t2.display();
51     cout << "Total : "; result.display();
52
53     return 0;
54 }

```


Chapter 4

Operator Overloading

Question [2083 Baishakh] Q4

Problem: Explain different ways of how binary operator overloading is performed? Using a class to model a 3x3 matrix, overload the + and - operator to add and subtract two matrices.

Solution: Ways of Binary Operator Overloading: Binary operators operate on two operands. In C++, they can be overloaded in two ways:

- **Using Member Function:** When a binary operator is overloaded as a member function, it takes only one explicit argument (the right operand). The left operand is the object that invokes the function (implicitly passed via `this` pointer).
- **Using Friend / Non-Member Function:** When overloaded as a friend function, it takes two explicit arguments (the left and right operands). This is useful when the left operand is not an object of the class (e.g., `int + Object`).

C++ Program (Matrix Addition and Subtraction):

```
1 #include <iostream>
2 using namespace std;
3
4 class Matrix {
5 private:
6     int mat[3][3];
7 public:
8     void input() {
9         cout << "Enter 3x3 matrix elements:\n";
10        for (int i = 0; i < 3; i++)
11            for (int j = 0; j < 3; j++)
12                cin >> mat[i][j];
13    }
14
15    void display() {
16        for (int i = 0; i < 3; i++) {
17            for (int j = 0; j < 3; j++)
18                cout << mat[i][j] << " ";
19            cout << endl;
20        }
21    }
22
23    // Overloading + operator
24    Matrix operator+(Matrix m) {
25        Matrix temp;
26        for (int i = 0; i < 3; i++)
27            for (int j = 0; j < 3; j++)
28                temp.mat[i][j] = mat[i][j] + m.mat[i][j];
29        return temp;
30    }
```

```

30     }
31
32     // Overloading - operator
33     Matrix operator-(Matrix m) {
34         Matrix temp;
35         for (int i = 0; i < 3; i++)
36             for (int j = 0; j < 3; j++)
37                 temp.mat[i][j] = mat[i][j] - m.mat[i][j];
38         return temp;
39     }
40 };
41
42 int main() {
43     Matrix m1, m2, m3;
44     cout << "Matrix 1:\n"; m1.input();
45     cout << "Matrix 2:\n"; m2.input();
46
47     cout << "\nAddition (m1 + m2):\n";
48     m3 = m1 + m2;
49     m3.display();
50
51     cout << "\nSubtraction (m1 - m2):\n";
52     m3 = m1 - m2;
53     m3.display();
54
55     return 0;
56 }

```

Question [2082 Bhadra] Q3

Problem: What is an operator overloading? Write down the syntax for overloading binary operator non-member operator function in a class. Write a program to perform addition, subtraction, multiplication and division with complex objects with real and imaginary members.

Solution: Operator Overloading: Operator overloading is a compile-time polymorphism feature in C++ that allows developers to redefine the meaning of built-in operators (like +, -, *, /) when they are applied to user-defined data types (objects).

Syntax for Non-Member (Friend) Binary Operator Overloading:

```

1 friend ReturnType operator+(const ClassName& op1, const ClassName& op2);

```

C++ Program (Complex Numbers):

```

1 #include <iostream>
2 using namespace std;
3
4 class Complex {
5 private:
6     float real, imag;
7 public:
8     Complex(float r = 0, float i = 0) : real(r), imag(i) {}
9
10    // Non-member operator overloads
11    friend Complex operator+(Complex c1, Complex c2);
12    friend Complex operator-(Complex c1, Complex c2);
13    friend Complex operator*(Complex c1, Complex c2);
14    friend Complex operator/(Complex c1, Complex c2);
15
16    void display() {
17        cout << real << (imag >= 0 ? "+" : "") << imag << "i" << endl;
18    }
19 };

```

```

20
21 Complex operator+(Complex c1, Complex c2) {
22     return Complex(c1.real + c2.real, c1.imag + c2.imag);
23 }
24
25 Complex operator-(Complex c1, Complex c2) {
26     return Complex(c1.real - c2.real, c1.imag - c2.imag);
27 }
28
29 Complex operator*(Complex c1, Complex c2) {
30     return Complex((c1.real * c2.real) - (c1.imag * c2.imag),
31                   (c1.real * c2.imag) + (c1.imag * c2.real));
32 }
33
34 Complex operator/(Complex c1, Complex c2) {
35     float denom = (c2.real * c2.real) + (c2.imag * c2.imag);
36     return Complex(((c1.real * c2.real) + (c1.imag * c2.imag)) / denom,
37                   ((c1.imag * c2.real) - (c1.real * c2.imag)) / denom);
38 }
39
40 int main() {
41     Complex c1(4, 2), c2(2, 1), result;
42
43     result = c1 + c2; cout << "Addition: "; result.display();
44     result = c1 - c2; cout << "Subtraction: "; result.display();
45     result = c1 * c2; cout << "Multiplication: "; result.display();
46     result = c1 / c2; cout << "Division: "; result.display();
47
48     return 0;
49 }

```

Question [2082 Baishakh] Q4

Problem: What are the rules for operator overloading? Write a program to create a class “time” with data members — hour, minute and seconds and then overload post increment operator for increment in seconds.

Solution: Rules for Operator Overloading:

- Only existing C++ operators can be overloaded. New operators cannot be created.
- The overloaded operator must have at least one operand that is of a user-defined type.
- We cannot change the basic meaning of an operator (e.g., cannot make + perform subtraction).
- Overloaded operators follow the syntax rules of the original operators (precedence and associativity cannot be changed).
- Some operators cannot be overloaded: ., .*, ::, ?:, sizeof.

C++ Program (Time Post-Increment):

```

1 #include <iostream>
2 using namespace std;
3
4 class Time {
5 private:
6     int hour, minute, second;
7 public:
8     Time(int h = 0, int m = 0, int s = 0) : hour(h), minute(m), second(s) {}
9

```

```

10 // Post-increment operator requires an int argument to distinguish from
    pre-increment
11 Time operator++(int) {
12     Time temp = *this; // Store original state
13     second++;
14     if (second >= 60) {
15         second = 0;
16         minute++;
17         if (minute >= 60) {
18             minute = 0;
19             hour++;
20         }
21     }
22     return temp; // Return original state
23 }
24
25 void display() {
26     cout << hour << ":" << minute << ":" << second << endl;
27 }
28 };
29
30 int main() {
31     Time t1(10, 59, 59);
32
33     cout << "Original time: ";
34     t1.display();
35
36     Time t2 = t1++; // Post-increment
37
38     cout << "Time after post-increment (t1): ";
39     t1.display();
40
41     cout << "Value assigned to t2 (old value): ";
42     t2.display();
43
44     return 0;
45 }

```

Question [2081 Ashwin] Q4

Problem: How is operator overloading used in case of binary operator? List down the rules of operator overloading. Create a class Length that has two data members meter and centimeter. Write a program to compare the length of two objects of this class by overloading ‘<’ operator. Use constructor to initialize data members.

Solution: Binary Operator Overloading: As described in [2083 Baishakh Q4], binary operators require two operands. When overloaded as a member function, one is implicit (**this**) and the other is explicit. When overloaded as a friend, both are explicit. The rules for operator overloading are detailed in [2082 Baishakh Q4].

C++ Program (Length Comparison):

```

1 #include <iostream>
2 using namespace std;
3
4 class Length {
5 private:
6     int meter, centimeter;
7 public:
8     Length(int m = 0, int cm = 0) {
9         meter = m;
10        centimeter = cm;
11        // Normalize

```

```

12     meter += centimeter / 100;
13     centimeter %= 100;
14 }
15
16 // Overloading '>' operator
17 bool operator>(Length l) {
18     int totalCm1 = meter * 100 + centimeter;
19     int totalCm2 = l.meter * 100 + l.centimeter;
20     return totalCm1 > totalCm2;
21 }
22
23 void display() {
24     cout << meter << "m " << centimeter << "cm";
25 }
26 };
27
28 int main() {
29     Length l1(5, 120); // Normalizes to 6m 20cm
30     Length l2(4, 250); // Normalizes to 6m 50cm
31
32     cout << "Length 1: "; l1.display(); cout << endl;
33     cout << "Length 2: "; l2.display(); cout << endl;
34
35     if (l1 > l2) {
36         cout << "Length 1 is greater than Length 2." << endl;
37     } else {
38         cout << "Length 1 is not greater than Length 2." << endl;
39     }
40
41     return 0;
42 }

```


Chapter 5

Inheritance

Question [2083 Baishakh] Q5

Problem: What is the difference between private and protected access specifier? Define function overriding. Write a program with a class to represent cricketer that has data members to represent name, age and number of matches played. From this class derive two classes bowler and batsman. The bowler class should have number of wickets as data member and batsman class should have number of runs and number of centuries as data members. Implement the concept of function overriding in your program and also mention the type of inheritance involved in the program.

Solution: Difference between Private and Protected:

- **Private:** Members declared as private are accessible only within the class itself. They cannot be accessed by derived classes.
- **Protected:** Members declared as protected are accessible within the class itself and also by derived (child) classes. They are inaccessible from outside the class hierarchy.

Function Overriding: Function overriding occurs when a derived class provides a specific implementation of a function that is already provided by its base class. The function in the derived class must have the same name, return type, and parameters as the one in the base class.

Type of Inheritance: Hierarchical Inheritance (one base class, multiple derived classes).

C++ Program:

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class Cricketer {
6 protected:
7     string name;
8     int age;
9     int matches_played;
10 public:
11     Cricketer(string n, int a, int m) : name(n), age(a), matches_played(m) {}
12
13     // Virtual function to enable overriding
14     virtual void display() {
15         cout << "Name: " << name << ", Age: " << age << ", Matches: " <<
16         matches_played << endl;
17     }
18 };
19
20 class Bowler : public Cricketer {
21 private:
```

```

21     int wickets;
22 public:
23     Bowler(string n, int a, int m, int w) : Cricketer(n, a, m), wickets(w) {}
24
25     void display() override {
26         cout << "[Bowler] Name: " << name << ", Age: " << age
27             << ", Matches: " << matches_played << ", Wickets: " << wickets
28             << endl;
29     };
30
31 class Batsman : public Cricketer {
32 private:
33     int runs;
34     int centuries;
35 public:
36     Batsman(string n, int a, int m, int r, int c) : Cricketer(n, a, m), runs(
37         r), centuries(c) {}
38
39     void display() override {
40         cout << "[Batsman] Name: " << name << ", Age: " << age
41             << ", Matches: " << matches_played << ", Runs: " << runs
42             << ", Centuries: " << centuries << endl;
43     };
44
45 int main() {
46     Bowler b("Sandeep", 23, 50, 100);
47     Batsman bt("Paras", 35, 100, 3000, 5);
48
49     // Function overriding in action
50     Cricketer* ptr;
51
52     ptr = &b;
53     ptr->display(); // Calls Bowler's display
54
55     ptr = &bt;
56     ptr->display(); // Calls Batsman's display
57
58     return 0;
59 }

```

Question [2082 Bhadra] Q4

Problem: Explain different forms of inheritance with an example for each. Create a class person with data members to identify a person and inherit it to create a class student with appropriate members. Write a complete program showing function overriding to make your program meaningful.

Solution: Forms of Inheritance:

- **Single Inheritance:** One derived class inherits from one base class.
- **Multiple Inheritance:** One derived class inherits from multiple base classes.
- **Hierarchical Inheritance:** Multiple derived classes inherit from a single base class.
- **Multilevel Inheritance:** A derived class inherits from another derived class.
- **Hybrid (Multipath) Inheritance:** A combination of two or more types of inheritance.

C++ Program (Person and Student):

```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class Person {
6 protected:
7     string name;
8     string address;
9 public:
10    Person(string n, string a) : name(n), address(a) {}
11
12    virtual void displayInfo() {
13        cout << "Person Name: " << name << ", Address: " << address << endl;
14    }
15 };
16
17 class Student : public Person {
18 private:
19     int rollNo;
20     string course;
21 public:
22    Student(string n, string a, int r, string c) : Person(n, a), rollNo(r),
        course(c) {}
23
24    // Function overriding
25    void displayInfo() override {
26        cout << "Student Name: " << name << ", Address: " << address
27            << ", Roll No: " << rollNo << ", Course: " << course << endl;
28    }
29 };
30
31 int main() {
32    Person p("John Doe", "New York");
33    Student s("Alice", "London", 101, "Computer Engineering");
34
35    Person* ptr = &p;
36    ptr->displayInfo();
37
38    ptr = &s;
39    ptr->displayInfo(); // Overridden function called
40
41    return 0;
42 }

```

Question [2082 Baishakh] Q5

Problem: Define inheritance in OOP. Give a suitable example to show how function overriding differ from function overloading? Write a program to show constructor invocation order in multilevel inheritance.

Solution: Inheritance: Inheritance is the process of creating a new class (derived class) from an existing class (base class). The derived class inherits attributes and methods of the base class.

Function Overriding vs Function Overloading:

- **Overloading:** Multiple functions with the same name but different parameters in the same scope. (Compile-time polymorphism).
- **Overriding:** Function in a derived class with the exact same name and signature as a virtual function in the base class. (Run-time polymorphism).

C++ Program (Constructor Invocation Order):

```

1 #include <iostream>
2 using namespace std;
3
4 class Grandparent {
5 public:
6     Grandparent() { cout << "Grandparent constructor called." << endl; }
7     ~Grandparent() { cout << "Grandparent destructor called." << endl; }
8 };
9
10 class Parent : public Grandparent {
11 public:
12     Parent() { cout << "Parent constructor called." << endl; }
13     ~Parent() { cout << "Parent destructor called." << endl; }
14 };
15
16 class Child : public Parent {
17 public:
18     Child() { cout << "Child constructor called." << endl; }
19     ~Child() { cout << "Child destructor called." << endl; }
20 };
21
22 int main() {
23     cout << "Creating Child object:" << endl;
24     {
25         Child c;
26     }
27     cout << "Child object destroyed." << endl;
28     return 0;
29 }

```

Output shows that constructors are invoked from Base to Derived (Grandparent → Parent → Child), while destructors are invoked from Derived to Base.

Question [2081 Ashwin] Q5

Problem: What is IS-A relation and HAS-A relationship? What do you mean by multipath inheritance? Explain how ambiguity due to multipath inheritance could be resolved.

Solution: IS-A vs HAS-A Relationship:

- **IS-A Relationship:** This represents Inheritance. For example, a Car IS-A Vehicle. It is implemented using public inheritance.
- **HAS-A Relationship:** This represents Composition/Aggregation. For example, a Car HAS-A Engine. It is implemented by including an object of one class as a member of another class.

Multipath Inheritance: Multipath inheritance occurs when a class is derived from two or more classes that share a common base class. This forms a diamond-shaped inheritance hierarchy (Diamond Problem).

Ambiguity Resolution: When the derived class accesses members of the common base class, ambiguity arises because two copies of the base class exist in the derived class. This is resolved using **Virtual Inheritance** (virtual base class).

```

1 #include <iostream>
2 using namespace std;
3
4 class CommonBase {
5 public:
6     int x;
7 };
8

```

```

9 // Virtual inheritance prevents duplicate CommonBase
10 class Derived1 : virtual public CommonBase {};
11 class Derived2 : virtual public CommonBase {};
12
13 class FinalDerived : public Derived1, public Derived2 {
14 public:
15     void show() {
16         x = 10; // No ambiguity because of virtual inheritance
17         cout << "Value: " << x << endl;
18     }
19 };
20
21 int main() {
22     FinalDerived fd;
23     fd.show();
24     return 0;
25 }

```


Chapter 6

Virtual Functions and Polymorphism

Question [2083 Baishakh] Q6

Problem: Why do you need virtual destructor? Distinguish between function overloading and overriding with example.

Solution: Why virtual destructor is needed: When we delete a derived class object using a pointer to a base class, the base class's destructor is called by default, leaving the derived class's resources unreleased (memory leak). Making the base class destructor **virtual** ensures that the destructor of the derived class is called first, followed by the base class's destructor, ensuring proper cleanup.

Function Overloading vs Overriding:

- **Function Overloading:** Two or more functions in the same scope have the same name but different parameters. (Compile-time polymorphism).
- **Function Overriding:** A derived class has a function with the exact same name and signature as a virtual function in the base class. (Run-time polymorphism).

Example:

```
1 #include <iostream>
2 using namespace std;
3
4 class Base {
5 public:
6     // Overloading: same name, different parameters
7     void display(int a) { cout << "Base display(int): " << a << endl; }
8     void display(double a) { cout << "Base display(double): " << a << endl; }
9
10    // Virtual function for overriding
11    virtual void show() { cout << "Base show()" << endl; }
12
13    // Virtual destructor
14    virtual ~Base() { cout << "Base Destructor" << endl; }
15 };
16
17 class Derived : public Base {
18 public:
19     // Overriding: exact same signature
20     void show() override { cout << "Derived show()" << endl; }
21
22     ~Derived() { cout << "Derived Destructor" << endl; }
23 };
24
```

```

25 int main() {
26     Base* bPtr = new Derived();
27     bPtr->display(5);    // Overloading
28     bPtr->display(5.5);  // Overloading
29
30     bPtr->show();        // Overriding (calls Derived's show)
31
32     delete bPtr;        // Calls Derived destructor then Base destructor
33     return 0;
34 }

```

Question [2082 Bhadra] Q5

Problem: Discuss about late binding and early binding. Write a program to create a classes Employee. Now create a class manager which is derived from employee. Show the use of virtual function with suitable data members and member functions.

Solution: Early Binding (Static Binding): In early binding, the compiler matches the function call with the correct function definition at compile time. It is faster but less flexible. Function overloading and normal member function calls are examples.

Late Binding (Dynamic Binding): In late binding, the function call is resolved at runtime based on the actual type of the object pointed to by a pointer/reference. It is implemented using virtual functions and provides flexibility (run-time polymorphism).

C++ Program:

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Employee {
6  protected:
7      string name;
8      double salary;
9  public:
10     Employee(string n, double s) : name(n), salary(s) {}
11
12     // Virtual function for late binding
13     virtual void calculateBonus() {
14         cout << "Employee " << name << " bonus: " << salary * 0.05 << endl;
15     }
16 };
17
18 class Manager : public Employee {
19 private:
20     double allowance;
21 public:
22     Manager(string n, double s, double a) : Employee(n, s), allowance(a) {}
23
24     // Overriding virtual function
25     void calculateBonus() override {
26         double totalSalary = salary + allowance;
27         cout << "Manager " << name << " bonus: " << totalSalary * 0.10 <<
endl;
28     }
29 };
30
31 int main() {
32     Employee emp("Raj", 50000);
33     Manager mgr("Sita", 80000, 20000);
34
35     Employee* ptr;
36

```

```

37     ptr = &emp;
38     ptr->calculateBonus(); // Early binding conceptually, but handled
                             dynamically
39
40     ptr = &mgr;
41     ptr->calculateBonus(); // Late binding: calls Manager's function
42
43     return 0;
44 }

```

Question [2082 Baishakh] Q6

Problem: What is virtual function? Define pure virtual function with suitable example.

Solution: Virtual Function: A virtual function is a member function in the base class that is declared using the keyword `virtual`. It is overridden in the derived class, and ensures that the correct function is called for an object, regardless of the type of reference (or pointer) used for function call (Late Binding).

Pure Virtual Function: A pure virtual function is a virtual function in C++ for which we do not write any implementation in the base class. It is declared by assigning 0 in its declaration. A class containing at least one pure virtual function becomes an **abstract class** and cannot be instantiated.

Example:

```

1  #include <iostream>
2  using namespace std;
3
4  // Abstract class
5  class Shape {
6  public:
7      // Pure virtual function
8      virtual void draw() = 0;
9  };
10
11 class Circle : public Shape {
12 public:
13     void draw() override {
14         cout << "Drawing a Circle" << endl;
15     }
16 };
17
18 class Rectangle : public Shape {
19 public:
20     void draw() override {
21         cout << "Drawing a Rectangle" << endl;
22     }
23 };
24
25 int main() {
26     // Shape s; // Error: Cannot instantiate abstract class
27
28     Shape* ptr1 = new Circle();
29     Shape* ptr2 = new Rectangle();
30
31     ptr1->draw();
32     ptr2->draw();
33
34     delete ptr1;
35     delete ptr2;
36     return 0;
37 }

```

Question [2081 Ashwin] Q6

Problem: What do you mean by abstract class? WAP to create two classes employee and student. Now create a class manager which is derived from two base classes employee and student and a class secretary derived from employee. Show the use of virtual destructor.

Solution: Abstract Class: An abstract class is a class that has at least one pure virtual function. Its purpose is to act as a base class and provide a blueprint for derived classes. You cannot create objects of an abstract class.

C++ Program:

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class Employee {
6 protected:
7     int empId;
8 public:
9     Employee(int id) : empId(id) {}
10
11     virtual void display() {
12         cout << "Employee ID: " << empId << endl;
13     }
14
15     // Virtual destructor
16     virtual ~Employee() {
17         cout << "Employee Destructor Called" << endl;
18     }
19 };
20
21 class Student {
22 protected:
23     string degree;
24 public:
25     Student(string d) : degree(d) {}
26
27     virtual void display() {
28         cout << "Student Degree: " << degree << endl;
29     }
30
31     virtual ~Student() {
32         cout << "Student Destructor Called" << endl;
33     }
34 };
35
36 // Multiple Inheritance
37 class Manager : public Employee, public Student {
38 public:
39     Manager(int id, string d) : Employee(id), Student(d) {}
40
41     void display() override {
42         cout << "Manager -> ";
43         Employee::display();
44         Student::display();
45     }
46
47     ~Manager() {
48         cout << "Manager Destructor Called" << endl;
49     }
50 };
51
52 // Single Inheritance
53 class Secretary : public Employee {
```

```

54 public:
55     Secretary(int id) : Employee(id) {}
56
57     void display() override {
58         cout << "Secretary -> ";
59         Employee::display();
60     }
61
62     ~Secretary() {
63         cout << "Secretary Destructor Called" << endl;
64     }
65 };
66
67 int main() {
68     cout << "--- Testing Virtual Destructor ---" << endl;
69
70     Employee* emp1 = new Manager(101, "MBA");
71     Employee* emp2 = new Secretary(102);
72
73     emp1->display();
74     emp2->display();
75
76     cout << "\nDeleting Manager via Employee pointer:" << endl;
77     delete emp1; // Will call Manager's then Employee's destructor
78
79     cout << "\nDeleting Secretary via Employee pointer:" << endl;
80     delete emp2; // Will call Secretary's then Employee's destructor
81
82     return 0;
83 }

```


Chapter 7

Stream Computation and File Handling

Question [2083 Baishakh] Q7

Problem: Explain how abnormal situations are handled in I/O process. Write a program that writes text to source file after reading from the user. Copy the content from source file, change the case of letters to uppercase if they are in lowercase and then store in destination file. Also, display the content of destination file too.

Solution: Abnormal situations in I/O Process: C++ streams maintain error state flags to handle abnormal situations (like end-of-file reached, formatting errors, or disk failures):

- eofbit: Set to 1 when end-of-file is reached. (`ios::eof()`)
- failbit: Set to 1 when a formatting error occurs, but characters are not lost. (`ios::fail()`)
- badbit: Set to 1 when a fatal error occurs (like physical disk error). (`ios::bad()`)
- goodbit: Set to 1 when no error has occurred. (`ios::good()`)
- The `clear()` function can be used to reset the error state flags.

C++ Program:

```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4 #include <cctype>
5 using namespace std;
6
7 int main() {
8     string text;
9     cout << "Enter text to write to source file: ";
10    getline(cin, text);
11
12    // Write to source file
13    ofstream outFile("source.txt");
14    if (!outFile) {
15        cerr << "Error creating source file!" << endl;
16        return 1;
17    }
18    outFile << text;
19    outFile.close();
20
21    // Read from source and write to destination
22    ifstream inFile("source.txt");
```

```

23     ofstream destFile("destination.txt");
24     if (!inFile || !destFile) {
25         cerr << "Error opening files!" << endl;
26         return 1;
27     }
28
29     char ch;
30     while (inFile.get(ch)) {
31         if (islower(ch)) {
32             ch = toupper(ch);
33         }
34         destFile.put(ch);
35     }
36     inFile.close();
37     destFile.close();
38
39     // Display destination file content
40     cout << "\nContent of destination file:\n";
41     ifstream readDest("destination.txt");
42     while (readDest.get(ch)) {
43         cout << ch;
44     }
45     readDest.close();
46     cout << endl;
47
48     return 0;
49 }

```

Question [2082 Bhadra] Q6

Problem: What are the different types of file modes used in C++? Write a program with a class to represent an employee having data members to represent name, employee Id, age and salary. Add record of 10 employees in a file and display them in a console. Also search a specific record of an employee using employee Id as a key from the user input.

Solution: Types of File Modes in C++:

- `ios::in`: Open for reading (default for `ifstream`).
- `ios::out`: Open for writing (default for `ofstream`). Overwrites existing file.
- `ios::app`: Append mode. All output is appended to the end of the file.
- `ios::ate`: Seek to the end of stream immediately after opening.
- `ios::trunc`: Truncate the file (empty it) before opening.
- `ios::binary`: Open file in binary mode rather than text mode.

C++ Program:

```

1 #include <iostream>
2 #include <fstream>
3 #include <string>
4 using namespace std;
5
6 class Employee {
7 public:
8     char name[50];
9     int empId;
10    int age;
11    double salary;
12

```

```

13     void input() {
14         cout << "Enter Name, ID, Age, Salary: ";
15         cin >> name >> empId >> age >> salary;
16     }
17     void display() {
18         cout << "ID: " << empId << ", Name: " << name
19             << ", Age: " << age << ", Salary: " << salary << endl;
20     }
21 };
22
23 int main() {
24     fstream file;
25     Employee emp;
26
27     // Writing 10 records
28     file.open("employees.dat", ios::out | ios::binary);
29     cout << "Enter details for 10 employees:\n";
30     for(int i = 0; i < 10; i++) {
31         cout << "Employee " << i+1 << ":\n";
32         emp.input();
33         file.write((char*)&emp, sizeof(emp));
34     }
35     file.close();
36
37     // Reading and displaying
38     file.open("employees.dat", ios::in | ios::binary);
39     cout << "\nAll Employees:\n";
40     while(file.read((char*)&emp, sizeof(emp))) {
41         emp.display();
42     }
43     file.close();
44
45     // Searching
46     int searchId;
47     bool found = false;
48     cout << "\nEnter Employee ID to search: ";
49     cin >> searchId;
50
51     file.open("employees.dat", ios::in | ios::binary);
52     while(file.read((char*)&emp, sizeof(emp))) {
53         if(emp.empId == searchId) {
54             cout << "Record Found:\n";
55             emp.display();
56             found = true;
57             break;
58         }
59     }
60     if(!found) {
61         cout << "Employee with ID " << searchId << " not found." << endl;
62     }
63     file.close();
64
65     return 0;
66 }

```

(Note: Using char arrays instead of string objects inside classes intended to be written to binary files is essential because *string* class maintains dynamic pointers, which causes memory faults when written directly to files.)

Question [2082 Baishakh] Q7

Problem: How random access of data in file can be handled? Write a program to store name, roll_no, address and batch of 'n' students in a file. Also add a feature to search the

student information by roll_no.

Solution: Handling Random Access in C++: C++ file streams provide methods to move the file pointers randomly:

- `seekg(offset, direction)`: Moves the **get** (read) pointer.
- `seekp(offset, direction)`: Moves the **put** (write) pointer.
- `tellg()`: Returns current position of the **get** pointer.
- `tellp()`: Returns current position of the **put** pointer.
- **Direction** can be `ios::beg` (beginning), `ios::cur` (current), or `ios::end` (end of file).

C++ Program:

```
1 #include <iostream>
2 #include <fstream>
3 using namespace std;
4
5 class Student {
6 public:
7     char name[50];
8     int roll_no;
9     char address[50];
10    int batch;
11
12    void input() {
13        cout << "Name: "; cin >> name;
14        cout << "Roll No: "; cin >> roll_no;
15        cout << "Address: "; cin >> address;
16        cout << "Batch: "; cin >> batch;
17    }
18    void display() {
19        cout << "Roll No: " << roll_no << ", Name: " << name
20            << ", Address: " << address << ", Batch: " << batch << endl;
21    }
22 };
23
24 int main() {
25     int n;
26     cout << "Enter number of students (n): ";
27     cin >> n;
28
29     ofstream outFile("students.dat", ios::binary);
30     Student s;
31     for (int i = 0; i < n; i++) {
32         cout << "Student " << i+1 << ":\n";
33         s.input();
34         outFile.write((char*)&s, sizeof(s));
35     }
36     outFile.close();
37
38     int r;
39     cout << "\nEnter Roll No to search: ";
40     cin >> r;
41
42     ifstream inFile("students.dat", ios::binary);
43     bool found = false;
44
45     // We could use seekg to jump if files were ordered, but sequential
46     search is used
```

```

46     while (inFile.read((char*)&s, sizeof(s))) {
47         if (s.roll_no == r) {
48             cout << "Found:\n";
49             s.display();
50             found = true;
51             break;
52         }
53     }
54
55     if (!found) cout << "Not found." << endl;
56     inFile.close();
57
58     return 0;
59 }

```

Question [2081 Ashwin] Q7

Problem: Explain different types of file access. Write a program for library to store information of different books (as least 5 items) in a file and display them on console.

Solution: Different Types of File Access:

- **Sequential Access:** Data is read or written sequentially from the beginning to the end. You cannot jump directly to a specific record. Useful for text files.
- **Random Access:** Allows reading or writing data at any specific location in the file using file pointers (`seekg`, `seekp`). Useful for database-like binary files containing fixed-size records.

C++ Program:

```

1  #include <iostream>
2  #include <fstream>
3  using namespace std;
4
5  class Book {
6  public:
7      int id;
8      char title[50];
9      char author[50];
10     float price;
11
12     void input() {
13         cout << "ID: "; cin >> id;
14         cout << "Title: "; cin >> title;
15         cout << "Author: "; cin >> author;
16         cout << "Price: "; cin >> price;
17     }
18     void display() {
19         cout << "ID: " << id << " | Title: " << title
20             << " | Author: " << author << " | Price: Rs." << price << endl;
21     }
22 };
23
24 int main() {
25     Book b;
26     ofstream outFile("library.dat", ios::binary);
27
28     cout << "Enter details for 5 books:\n";
29     for(int i = 0; i < 5; i++) {
30         cout << "Book " << i+1 << ":\n";
31         b.input();
32         outFile.write((char*)&b, sizeof(b));

```

```
33     }
34     outFile.close();
35
36     cout << "\n--- Books in Library ---\n";
37     ifstream inFile("library.dat", ios::binary);
38     while (inFile.read((char*)&b, sizeof(b))) {
39         b.display();
40     }
41     inFile.close();
42
43     return 0;
44 }
```

Chapter 8

Templates and STL

Question [2083 Baishakh] Q8

Problem: Why are templates in C++ referred to as a form of generic programming? What are containers and iterators in STL? Briefly explain default arguments with Class Template with suitable example.

Solution: Templates as Generic Programming: Templates allow us to write a single code body that can work with any data type. Instead of writing overloaded functions for `int`, `float`, `char`, etc., we write a single template, and the compiler generates the specific versions as needed. This approach of writing code independent of data types is called generic programming.

Containers and Iterators in STL:

- **Containers:** Objects that store collections of other objects (data). Examples: `vector`, `list`, `map`, `set`, `stack`.
- **Iterators:** Objects that point to the elements in a container, acting like pointers. They are used to traverse the elements of a container (e.g., `vector<int>::iterator`).

Default Arguments with Class Template: Just like functions can have default arguments, class templates can have default template arguments. If the user does not specify a data type when creating an object, the default type is used.

Example:

```
1 #include <iostream>
2 using namespace std;
3
4 // Class template with a default argument (int)
5 template <class T = int>
6 class MyContainer {
7 private:
8     T value;
9 public:
10    MyContainer(T val) : value(val) {}
11    void display() {
12        cout << "Value: " << value << endl;
13    }
14 };
15
16 int main() {
17     // Uses default type (int)
18     MyContainer<> obj1(100);
19     obj1.display();
20
21     // Explicitly uses double
22     MyContainer<double> obj2(99.99);
```

```

23     obj2.display();
24
25     return 0;
26 }

```

Question [2082 Bhadra] Q7

Problem: Show with an example how do we eliminate code redundancy using template. Give with example. WAP to read n numbers of any types and sort them on descending order. For sorting, you can use vector member function.

Solution: Eliminating Code Redundancy: Without templates, to swap two integers and two doubles, we must write two overloaded functions. This is redundant. A template function eliminates this by replacing the specific types with a generic placeholder (like T).

C++ Program (Sorting Vector Descending):

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm> // for sort
4  using namespace std;
5
6  // Template function to read, sort and display vector
7  template <class T>
8  void processSort() {
9      int n;
10     cout << "Enter the number of elements: ";
11     cin >> n;
12
13     vector<T> vec(n);
14     cout << "Enter " << n << " elements:\n";
15     for(int i = 0; i < n; i++) {
16         cin >> vec[i];
17     }
18
19     // Sort in descending order using greater<T>()
20     sort(vec.begin(), vec.end(), greater<T>());
21
22     cout << "Sorted in descending order:\n";
23     for(int i = 0; i < n; i++) {
24         cout << vec[i] << " ";
25     }
26     cout << "\n-----\n";
27 }
28
29 int main() {
30     cout << "Processing Integers:\n";
31     processSort<int>();
32
33     cout << "Processing Doubles:\n";
34     processSort<double>();
35
36     return 0;
37 }

```

Question [2082 Baishakh] Q8

Problem: “Templates in C++ enable generic programming, allowing the creation of reusable functions and classes that work with different data types within a single framework”. Give example to justify the given statement.

Solution: The statement describes how templates act as a blueprint. A single template definition can handle integers, floating-point numbers, and even user-defined objects, provided the operations inside the template are supported by those types.

Example Justification: Consider a generic `findMax` function. Instead of writing it multiple times for different types, we write it once:

```
1 #include <iostream>
2 using namespace std;
3
4 // Reusable function for ANY type
5 template <class T>
6 T findMax(T a, T b) {
7     return (a > b) ? a : b;
8 }
9
10 int main() {
11     // Works with int
12     cout << "Max int: " << findMax(10, 20) << endl;
13
14     // Works with double
15     cout << "Max double: " << findMax(5.5, 3.2) << endl;
16
17     // Works with char
18     cout << "Max char: " << findMax('A', 'Z') << endl;
19
20     return 0;
21 }
```

This single framework reduces code size, increases reusability, and is the core of generic programming.

Question [2081 Ashwin] Q8

Problem: Explain the use of default argument in template class. Write a suitable program to show the concept of function template. Explain in brief about stack in Standard Template Library.

Solution: Default Argument in Template Class: As explained in [2083 Baishakh Q8], default template arguments allow users to omit specifying a type if they are satisfied with the default type (e.g., `template <class T = int>`).

Function Template Concept: A function template defines a family of functions. The concept was shown in the `findMax` program above.

Stack in STL: `stack` is a container adapter in the C++ Standard Template Library (STL). It provides a Last-In-First-Out (LIFO) data structure.

- Elements are added and removed from the same end, called the "top".
- Key operations:
 - `push()`: Add an element to the top.
 - `pop()`: Remove the top element.
 - `top()`: Access the top element.
 - `empty()`: Check if the stack is empty.

```
1 #include <iostream>
2 #include <stack>
3 using namespace std;
4
5 int main() {
```

```
6     stack<int> s;
7     s.push(10);
8     s.push(20);
9     s.push(30);
10
11     cout << "Top element is: " << s.top() << endl; // 30
12     s.pop();
13     cout << "Top element after pop: " << s.top() << endl; // 20
14
15     return 0;
16 }
```

Chapter 9

Exception Handling

Question [2083 Baishakh] Q9

Problem: How is exception handling mechanism in C++ better than tradition techniques? Write a program to read name, roll number and marks that catches multiple exceptions using exception class if roll number is negative and marks greater than full marks. Also, handle the exceptions.

Solution: Why Exception Handling is better than Traditional Techniques:

Traditional error handling uses return codes (e.g., returning -1 on error) or global flags.

- **Separation of Logic:** Exceptions separate the error-handling code from the normal flow of the program, making the code cleaner.
- **Cannot be ignored:** Return codes can be ignored by the programmer, but unhandled exceptions crash the program, forcing the developer to address the issue.
- **Type-Safe:** Exceptions can carry objects (exception classes) that contain detailed information about the error.

C++ Program:

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 // Custom Exception Classes
6 class InvalidRollNumber {};
7 class InvalidMarks {};
8
9 class Student {
10 private:
11     string name;
12     int roll;
13     int marks;
14 public:
15     void input() {
16         cout << "Enter Name: ";
17         cin >> name;
18
19         cout << "Enter Roll Number: ";
20         cin >> roll;
21         if (roll < 0) {
22             throw InvalidRollNumber(); // Throw object of exception class
23         }
24
25         cout << "Enter Marks (out of 100): ";
26         cin >> marks;
```

```

27         if (marks > 100 || marks < 0) {
28             throw InvalidMarks();
29         }
30     }
31
32     void display() {
33         cout << "Record saved -> Name: " << name << ", Roll: " << roll << ",
34         Marks: " << marks << endl;
35     }
36 };
37
38 int main() {
39     Student s;
40     try {
41         s.input();
42         s.display();
43     }
44     catch (InvalidRollNumber) {
45         cerr << "Exception Caught: Roll number cannot be negative!" << endl;
46     }
47     catch (InvalidMarks) {
48         cerr << "Exception Caught: Marks must be between 0 and 100!" << endl;
49     }
50     catch (...) {
51         cerr << "Exception Caught: Unknown error occurred!" << endl;
52     }
53
54     return 0;
55 }

```

Question [2082 Bhadra] Q8

Problem: Explain exception along with their handling constructs. Develop a C++ program that shows multiple exception handling scenarios.

Solution: Exceptions: Exceptions are run-time anomalies or unusual conditions that a program may encounter during execution (e.g., divide by zero, array out of bounds, running out of memory).

Handling Constructs:

- **try:** A block of code that may generate an exception.
- **throw:** When a problem is detected, an exception is thrown using this keyword.
- **catch:** A block of code that catches and handles the exception thrown by the **try** block.

C++ Program (Multiple Exceptions):

```

1 #include <iostream>
2 using namespace std;
3
4 void test(int x) {
5     try {
6         if (x == 1) throw x;           // Throwing integer
7         else if (x == 2) throw 'x';    // Throwing character
8         else if (x == 3) throw 5.5;    // Throwing double
9         cout << "End of try block.\n";
10    }
11    catch (int i) {
12        cout << "Caught an integer: " << i << endl;
13    }
14    catch (char c) {

```

```

15     cout << "Caught a character: " << c << endl;
16 }
17 catch (double d) {
18     cout << "Caught a double: " << d << endl;
19 }
20 }
21
22 int main() {
23     cout << "Testing x == 1:\n"; test(1);
24     cout << "Testing x == 2:\n"; test(2);
25     cout << "Testing x == 3:\n"; test(3);
26     cout << "Testing x == 4:\n"; test(4); // No exception thrown
27
28     return 0;
29 }

```

Question [2082 Baishakh] Q9

Problem: What are exceptions in a program? Write a meaningful program that can handle multiple exceptions.

Solution: The definition of an exception and a meaningful program to handle multiple exceptions using different catch blocks has been detailed in [2082 Bhadra Q8]. An alternative way is to use a catch-all block (`catch(...)`) which catches any thrown exception that is not caught by preceding catch blocks.

Question [2081 Ashwin] Q9

Problem: Discuss steps taken during exception handling. Write a program to compute square root of a number. The input value must be tested for validity. If it is negative, an exception must be raised.

Solution: Steps in Exception Handling:

1. **Hit the Exception:** The program executes within a `try` block.
2. **Throwing the Exception:** When an error is detected, a `throw` statement is executed. This immediately terminates the current block and transfers control.
3. **Catching the Exception:** The runtime system searches for a `catch` block that can handle the type of exception thrown.
4. **Handling:** The code inside the matching `catch` block is executed to resolve the issue or notify the user.
5. **Resumption:** After the `catch` block executes, program execution continues with the statement immediately following the try-catch structure.

C++ Program (Square Root Validation):

```

1  #include <iostream>
2  #include <cmath>
3  using namespace std;
4
5  int main() {
6      double num;
7      cout << "Enter a number to find its square root: ";
8      cin >> num;
9
10     try {
11         if (num < 0) {

```

```
12         throw "Error: Cannot calculate square root of a negative number!"
13     ;
14     }
15     double result = sqrt(num);
16     cout << "Square root of " << num << " is " << result << endl;
17 }
18 catch (const char* msg) {
19     cerr << msg << endl;
20 }
21
22 cout << "Program execution continues gracefully..." << endl;
23 return 0;
24 }
```